SummarAI

Technical Report

Automated Meeting Summarization using Whisper + Claude

NLP Group Project . 2025-2026

Antonio . Bojana . Jo . Marti . Smaragda

1. Introduction

Meetings generate decisions and commitments, but most of that information is lost shortly after the call ends. People rely on informal notes, fragmented memory, or re-watching recordings — none of which scale. **SummarAI** is a web application that automates this capture: upload an English-language meeting recording and, within minutes, receive a concise written record structured around two outputs: a short prose summary and a set of bullet points covering key takeaways and action items.

The system chains two NLP components. First, a local speech recognition model (Whisper) converts audio to text. Second, a large language model (Claude Haiku 4.5) reads that transcript and produces structured output through a strict JSON schema. The web interface is minimal by design — the value is in the NLP pipeline, not the UI.

This report describes the motivation, the technical design, an empirical evaluation, and the failure modes documented during development.

2. Justification of Need

Meetings consume a large share of knowledge workers' time, yet much of their value is lost once the call ends. The problem is rarely the discussion itself but the follow-through: decisions made verbally go unrecorded, action items are forgotten, and participants who were absent have no reliable way to catch up. A written record fixes this, but taking accurate minutes by hand is tedious and routinely skipped.

Several tools exist to address this — Otter.ai, Fireflies.ai, Zoom AI Companion, and Microsoft Teams Copilot all offer some form of meeting transcription and summary. Their limitations are relevant to this project:

- **Cost and access:** all major tools require paid subscriptions; several are tied to specific video-conferencing platforms.
- **Output quality:** most tools produce a raw transcript or a continuous prose summary. They rarely separate *decisions* from *notes*, and action-item extraction is either absent or unreliable when responsibility is only implied.
- **Closed systems:** none of these tools expose the underlying model or prompt, making it impossible to understand or modify their behaviour.

SummarAI addresses the output-quality gap specifically. Its two-output design — a 2-sentence summary plus typed bullet points (decision / note / action item) — forces the model to make an explicit distinction that conversational summaries routinely blur.

3. Field Review

3.1 Automatic Speech Recognition (ASR)

The dominant open-source ASR model at the time of writing is OpenAI Whisper (Radford et al., 2022). Whisper is an encoder-decoder transformer trained on 680,000 hours of web audio, achieving near-human accuracy on clean English speech and robust performance on accented or lightly noisy audio. Its English-only variants (*tiny.en*, *base.en*, *small.en*) outperform the corresponding multilingual

models on English audio while being faster.

faster-whisper is a community re-implementation of Whisper using the CTranslate2 inference library. It achieves the same output as the original at 2–4× lower memory usage and measurably faster runtime, making it practical for a local CPU deployment without a GPU.

Commercial ASR alternatives (AssemblyAI, AWS Transcribe, Azure Cognitive Services) offer higher accuracy on telephony-quality audio and built-in speaker diarization, at per-minute costs that make them unsuitable for a course prototype.

3.2 Meeting Summarization and Information Extraction

LLM-based summarization has largely displaced earlier extractive and abstractive approaches (TextRank, BART, PEGASUS) for long-document tasks. Instruction-tuned models handle long, noisy meeting transcripts better than task-specific fine-tuned models, primarily because they can follow complex, multi-part instructions about output format.

Structured output — enforcing a JSON schema at the API level rather than prompt-engineering the model to emit valid JSON — eliminates parsing failures caused by model deviation from the expected format. Claude Haiku 4.5 was selected for this project on cost grounds: it is the fastest and cheapest model in the Claude 4 family while still following complex structured instructions reliably.

4. System Description

4.1 Architecture

SummarAI is a single-server web application. The backend is a Python FastAPI app (*src/app.py*). The frontend is a single-page vanilla JavaScript application (*src/static/index.html*). Meeting results can optionally be saved to a Supabase (PostgreSQL) database for project-level persistence and timeline views.

Category	Supported Extensions
Audio / Video	.mp4 · .mp3 · .wav · .m4a · .webm · .ogg · .flac
Text (skip ASR)	.txt · .md · .vtt · .srt

4.2 Two-Stage NLP Pipeline

Stage 1 — Speech-to-text. *faster-whisper* runs locally on CPU using the *base.en* model in int8 quantization. Voice activity detection (*vad_filter=True*) strips silence before transcription. The output is a plain string of the full meeting speech.

Stage 2 — Summarization and information extraction. The transcript is sent to Claude Haiku 4.5 with a system prompt that instructs the model to produce exactly three structured fields:

- **summary** — at most 2 short sentences stating the meeting's purpose and overall outcome.
- **key_takeaways** — an array of bullet objects, each with a *text* field (≤14 words) and a *type* field: *"decision"* (something agreed or set as policy) or *"note"* (any other important point). The instruction to be comprehensive is deliberate: the 2-sentence summary omits detail by design, so takeaways carry all substantive information.
- **action_items** — concrete tasks with *owner* and *deadline* populated only when actually stated; never inferred.

4.3 Performance Optimization

During development we measured end-to-end processing time on a 20-minute .m4a file on a standard laptop CPU. Latency was almost entirely in the ASR stage — Claude Haiku 4.5 returned in 3–6 seconds regardless of transcript length. We applied four zero-cost changes to the transcription stage:

Change	Rationale
base.en instead of multilingual base	English-only model is faster and more accurate for English audio
beam_size=1 (greedy decoding)	Removes beam search overhead (~2× faster) with negligible accuracy loss
cpu_threads=os.cpu_count()	Uses all available cores instead of the library default
language="en"	Skips the automatic language-detection pass

Configuration	Total Time	Transcript Length
Original (base, beam 5, auto-detect)	7 min 50 s	36,702 chars

Configuration	Total Time	Transcript Length
Optimized (base.en, greedy, multithread)	2 min 12 s	36,620 chars

This represents a **3.5× speedup (≈72% reduction) at zero additional cost**, with no measurable effect on downstream summary quality.

5. Custom Evaluation

5.1 Methodology

Our test set is **30 meetings from the AMI Meeting Corpus** (Carletta et al., 2006), a widely used research dataset of recorded meetings. AMI is annotated with human-written abstractive summaries, each split into ABSTRACT, DECISIONS, and ACTIONS sections. We use those human annotations as ground truth, avoiding the circularity of grading an LLM against answers we wrote ourselves.

We feed our own Whisper transcripts (not AMI's clean reference transcripts) as input, so scores reflect the full audio-to-summary path, including transcription noise. Two metrics are reported:

- **ROUGE-1 / 2 / L (F-measure):** our full output against the human reference. ROUGE is the standard meeting-summarization metric.
- **Action-item coverage (recall):** the fraction of human ACTIONS sentences that a predicted action item covers, counting a match when ROUGE-L F ≥ 0.30.

5.2 Results

Metric	Score
ROUGE-1 (F, avg)	0.335
ROUGE-2 (F, avg)	0.057
ROUGE-L (F, avg)	0.148
Action-item coverage (recall)	0.424 (28 / 66)

The summaries are coherent and stay on topic. ROUGE figures are modest primarily due to a deliberate **format mismatch**: our summary is two sentences by design, whereas an AMI reference is a ~200-word multi-section document. ROUGE rewards n-gram overlap, so a deliberately compressed output is penalised on form even when its content is accurate. A ROUGE-1 of 0.34 means our output shares roughly a third of its unigrams with a much longer human summary — a reasonable result for an output that is an order of magnitude shorter.

The most informative number is the **42% action-item coverage**: working end to end from raw audio, the system recovers roughly four of every ten action items a human annotator recorded.

5.3 Error Analysis

Two systematic factors explain most missed action items:

- **Representation gap.** AMI annotates actions by *role* (“the industrial designer will work on the design”), while SummarAI extracts them by *name*. The surface forms differ enough to fall below the overlap threshold.
- **Owner attribution / speaker identity.** Whisper produces no speaker labels. When responsibility is implied rather than named, the model leaves owner null rather than guessing — conservative and arguably correct, but it counts as an incomplete action item.

6. Failure Mode Analysis

The following cases were documented during development and evaluation.

1. Implied speaker identity

When responsibility is suggested rather than stated (“we should look into this”), the model correctly produces an action item with owner: null. This limits the utility of the action-item list for follow-up.

2. Whisper mishearing proper nouns

Names uncommon in Whisper's training data are occasionally transcribed incorrectly (e.g. Bojana transcribed as Janna). This propagates directly into incorrect owner attribution downstream.

3. Off-topic or non-meeting audio

Podcast-style recordings, long monologues, and panel discussions produce coherent summaries but correctly yield zero action items. Users uploading non-meeting content may be confused by the empty action-item list.

4. Heavily overlapping or noisy audio

Whisper's VAD filter removes silence effectively but does not separate overlapping speakers. In meetings where multiple participants speak simultaneously, the transcript can contain garbled sentences. Summary quality degrades noticeably.

5. Very long meetings near the character limit

Processing time scales linearly with transcript length. A 2-hour meeting can take 4–5 minutes on modest hardware. The LLM context window is not a bottleneck at current meeting lengths.

6. Implicit decisions

When a decision emerges through consensus rather than an explicit statement, the model sometimes tags it as note rather than decision. The distinction is genuinely ambiguous in natural speech.

7. Repetitive or circular discussions

In meetings where the same point is debated multiple times, the model sometimes produces duplicate or near-duplicate takeaways. The system prompt instructs comprehensiveness, which can work against deduplication.

7. Future Directions

Speaker diarization. Integrating automatic speaker labelling would directly solve the owner attribution problem. Libraries such as pyannote.audio provide open-source diarization that could be inserted between the Whisper ASR stage and the LLM stage.

Larger Whisper models. The base.en model was chosen for speed. On hardware with more RAM, small.en or medium.en would improve accuracy on accented speech, domain-specific vocabulary, and noisy audio.

Confidence and uncertainty signals. The current system does not communicate uncertainty. A useful extension would flag action items or decisions where the model's confidence is low.

Persistent project search. Adding full-text search over saved summaries and takeaways would make the meeting archive genuinely useful as an organizational memory tool.

Deployment. The local Whisper model rules out serverless hosting (Vercel, Lambda) due to memory and runtime constraints. A long-running FastAPI server on Render, Railway, or Fly.io would work without changes to the application code.

8. Use of AI Tools

Claude (Anthropic) was used throughout the project in several capacities:

- **Code generation and debugging:** the FastAPI backend, structured output schema, and Supabase integration were iteratively developed with Claude assistance.
- **System prompt engineering:** the system prompt was drafted collaboratively — initial versions generated by Claude Sonnet and refined through iterative testing on real transcripts.
- **Documentation:** sections of this report, the user manual, and the installation guide were drafted with Claude assistance and then reviewed and edited by team members.
- **Transcription (Whisper):** the ASR component is itself an AI model, used as a fixed inference component rather than interactively.

All code, prompts, and documentation were reviewed and edited by team members. No output was used without human review.

9. Conclusion

SummarAI demonstrates that a two-stage NLP pipeline — local ASR followed by a structured LLM call — can reliably transform an English meeting recording into an actionable written record. The system is functional, reproducible, and fast enough for practical use on consumer hardware.

The key finding from development was empirical: latency is dominated by transcription, not by the LLM, and a set of zero-cost Whisper optimizations reduced total processing time by 72% with no measurable loss in output quality. The key limitation is owner attribution, a problem that speaker diarization would directly address.

The two-output design (summary + bullets) proved to be the right abstraction. The 2-sentence summary is useful for quick orientation; the typed bullet list carries the substance. Separating decisions from notes forces the model to make a distinction that is genuinely useful for follow-up and accountability.

References

Carletta, J., et al. (2006). *The AMI meeting corpus: A pre-announcement.* Machine Learning for Multimodal Interaction (MLMI), 28–39.

Lewis, M., et al. (2020). *BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension.* Proceedings of ACL 2020.

Radford, A., et al. (2022). *Robust speech recognition via large-scale weak supervision.* arXiv preprint arXiv:2212.04356.

See, A., Liu, P. J., & Manning, C. D. (2017). *Get to the point: Summarization with pointer-generator networks.* Proceedings of ACL 2017.

SYSTRAN. (2023). *faster-whisper* [Computer software]. GitHub. https://github.com/SYSTRAN/faster-whisper

Vaswani, A., et al. (2017). *Attention is all you need.* Advances in Neural Information Processing Systems, 30.